Diseño correcto de una API REST

En este momento vamos a trabajar los fundamentos necesarios para construir una API REST bien diseñada utilizando Spring Boot.

El objetivo no es únicamente conseguir que la aplicación funcione. Debéis comprender y demostrar cómo diseñar una API mantenible, clara y coherente, aplicando buenas prácticas profesionales.

Cada grupo trabajará un aspecto concreto del diseño de APIs REST, preparará una pequeña demostración funcional y explicará al resto de la clase qué problema resuelve y cómo debe aplicarse correctamente.

Organización

La clase se dividirá en **5 grupos**.

Cada grupo tendrá asignado uno de los siguientes temas:

Grupo 1 – Diseño REST y recursos

Debéis investigar y explicar cómo se diseña correctamente una API REST.

Trabajaréis especialmente:

- Qué es un recurso.
- Cómo diseñar correctamente las URL.
- Uso de sustantivos en lugar de verbos.
- Uso de los métodos HTTP:
    - GET
    - POST
    - PUT
    - PATCH
    - DELETE
- Diferencia entre colección y recurso individual.
- Relaciones entre recursos.
- Concepto de API stateless.

Ejemplos que podéis analizar:

GET /clientes

GET /clientes/15

POST /clientes

PUT /clientes/15

DELETE /clientes/15

GET /clientes/15/pedidos

Demo

Debéis desarrollar una pequeña API que permita gestionar al menos un recurso.

La demo deberá mostrar:

- Obtener todos los elementos.
- Obtener un elemento por ID.
- Crear un elemento.
- Modificar un elemento.
- Eliminar un elemento.

Además, debéis enseñar al menos **dos ejemplos de endpoints mal diseñados** y explicar cómo los mejoraríais.

Grupo 2 – Códigos de estado HTTP y respuestas

Debéis investigar cómo debe informar una API al cliente del resultado de una operación.

Trabajaréis especialmente los códigos:

- 200 OK
- 201 Created
- 204 No Content
- 400 Bad Request
- 401 Unauthorized
- 403 Forbidden
- 404 Not Found
- 409 Conflict

- 500 Internal Server Error

Debéis explicar por qué una API no debería devolver siempre:

200 OK

aunque haya ocurrido un problema.

Demo

Desarrollad una API que produzca diferentes situaciones.

Por ejemplo:

GET /productos/5

si existe:

200 OK

si no existe:

404 Not Found

Al crear un elemento correctamente:

201 Created

Al intentar crear un elemento incorrecto:

400 Bad Request

Debéis demostrar al menos **cinco códigos HTTP diferentes**.

También deberéis explicar qué información debería contener el cuerpo de una respuesta de error.

Grupo 3 – DTO y separación entre API y base de datos

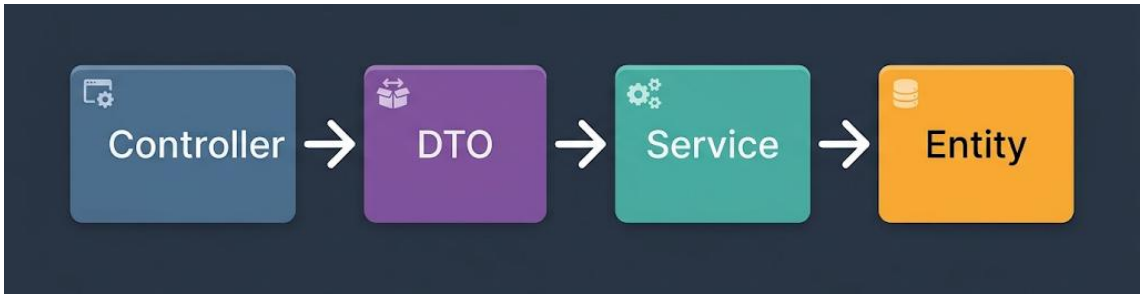
Debéis investigar qué son los **DTO (Data Transfer Object)** y por qué no es recomendable devolver directamente las entidades de la base de datos desde una API.

Partiremos del siguiente problema:



Debéis explicar los riesgos de este enfoque.

Después deberéis plantear:



Trabajaréis conceptos como:

- Entity.
- DTO.
- DTO de entrada.
- DTO de salida.
- Separación de responsabilidades.
- Ocultación de información.
- Acoplamiento entre API y persistencia.

Demo

Debéis crear una entidad que tenga algún campo que **no debería enviarse al cliente**.

Por ejemplo:

Usuario

- id

- nombre

- email

- password

Primero mostraréis qué ocurre si devolvemos directamente:

Usuario

y después crearéis:

UsuarioResponseDTO

que únicamente devuelva:

id

nombre

email

También deberéis mostrar la transformación:



y



Grupo 4 – Validación de datos

Debéis investigar cómo evitar que una API acepte datos incorrectos.

Por ejemplo, una API no debería permitir crear:

```
{
  "nombre": "",
  "email": "hola",
  "edad": -15
}
```

Trabajaréis la validación utilizando Bean Validation.

Algunas anotaciones que podéis investigar:

@NotNull

@NotBlank

@Size

@Min

@Max

@Email

@Positive

@Past

@Future

También deberéis trabajar con:

@Valid

Demo

Desarrollad un DTO de entrada con varias reglas de validación.

Por ejemplo:

```
public class UsuarioCreateDTO {


    @NotBlank
    private String nombre;


    @Email
    private String email;


    @Min(18)
    private Integer edad;


}
```

La aplicación debe:

- aceptar datos correctos;

- rechazar datos incorrectos;
- devolver un código adecuado;
- informar de qué campos contienen errores.

Debéis provocar durante la exposición diferentes errores de validación.

Grupo 5 – Tratamiento global de errores

Debéis investigar cómo gestionar correctamente los errores de una API.

Debéis analizar qué ocurre cuando aparecen excepciones como:

Cliente no encontrado

Producto no encontrado

Email duplicado

Stock insuficiente

Datos incorrectos

No queremos que cada controlador tenga continuamente código como:

```
try {


} catch (...) {


}
```

Trabajaréis:

@ControllerAdvice

@ExceptionHandler

También deberéis crear alguna excepción propia, por ejemplo:

ClienteNoEncontradoException

o:

ProductoNoEncontradoException

Demo obligatoria

Debéis crear un tratamiento centralizado de excepciones.

Por ejemplo:

GET /clientes/999

podría devolver:

```
{
  "status": 404,
  "error": "Not Found",
  "message": "No existe el cliente solicitado",
  "path": "/clientes/999"
}
```

Debéis provocar al menos **tres errores diferentes** y comprobar que todos utilizan una estructura de respuesta coherente.

Requisitos generales de la demo

Todos los grupos deberán desarrollar una pequeña API en **Spring Boot**.

No es necesario desarrollar una aplicación grande.

Podéis utilizar recursos sencillos como:

Usuarios

Productos

Libros

Películas

Videojuegos

Alumnos

Pedidos

Tareas

La aplicación debe permitir demostrar claramente vuestro concepto.

Podéis utilizar:

- Spring Boot.
- Spring Web.

- Spring Data JPA si es necesario.
- H2, MySQL o PostgreSQL.
- Postman, Bruno, Insomnia o Swagger para realizar las peticiones.

Estructura de la exposición

La exposición tendrá una duración máxima de **10 minutos**.

Debéis organizarla aproximadamente de la siguiente forma:

1. Presentación del problema

Explicad primero qué problema intentáis resolver.

No comencéis únicamente definiendo el concepto.

Por ejemplo:

¿Qué ocurre si nuestra API devuelve siempre código 200 aunque el producto no exista?

o:

¿Qué problema existe si devolvemos directamente nuestras entidades JPA?

2. Explicación del concepto

Explicad:

- qué es;
- para qué sirve;
- qué problema resuelve;
- cómo se utiliza.

3. Código

Mostrad únicamente las partes importantes.

No es necesario explicar línea por línea toda la aplicación.

4. Demo

La parte más importante de la exposición será la demostración.

Debéis ejecutar la API y realizar varias peticiones para demostrar que vuestra solución funciona.

5. Ejemplo incorrecto frente a ejemplo correcto

Siempre que sea posible debéis mostrar:

ANTES/DESPUÉS o MALA PRÁCTICA/BUENA PRÁCTICA

6. Conclusión

La exposición deberá terminar respondiendo brevemente:

- ¿Qué problema resuelve?
- ¿Qué ventajas aporta?
- ¿Qué errores frecuentes debemos evitar?
- ¿Cuándo merece la pena utilizarlo?

Normas importantes

No se valorará positivamente una presentación basada únicamente en teoría.

La exposición deberá incluir:

- explicación técnica
- código
- API funcionando
- peticiones reales
- ejemplo de mala práctica
- solución propuesta

Podéis utilizar inteligencia artificial como herramienta de apoyo, pero debéis ser capaces de **explicar y modificar todo el código que presentéis**.

Durante la exposición el profesor podrá preguntar por cualquier parte del código.

Si un miembro del grupo no sabe explicar el código desarrollado por su equipo, se considerará que esa parte del trabajo no ha sido comprendida.

Producto final

Cada grupo deberá entregar:

1. Proyecto completo de Spring Boot.
2. Código fuente.
3. Archivo README (en formato Markdown con imágenes) con:
    - nombre del grupo
    - tema trabajado
    - explicación breve
    - instrucciones para ejecutar el proyecto
    - endpoints disponibles
4. Material utilizado para la exposición.

Idea clave de esta primera ronda

El objetivo no es simplemente conseguir:

GET: funciona

POST: funciona

DELETE: funciona

Una API profesional no se diferencia de una API básica por tener más código, sino por estar **mejor diseñada**.